CS3 RESEARCH EXPERIENCE FOR TEACHERS

Introduction to Object Tracking Algorithms

Tracking by detection, the Kalman filter, SORT, and tracking metrics

Mehmet Kerem Turkcan

Center for Smart Streetscapes, Columbia University

Outline

1. **Multi-object tracking**: the task and the tracking-by-detection paradigm.
2. **The Kalman filter**: predicting where each bounding box will be in the next frame.
3. **SORT**: a simple online tracker, walked through in code.
4. **Evaluation**: MOTA, MOTP, IDF1, and HOTA.

PART 1

Multi-object tracking

Each tracked object keeps a persistent identity

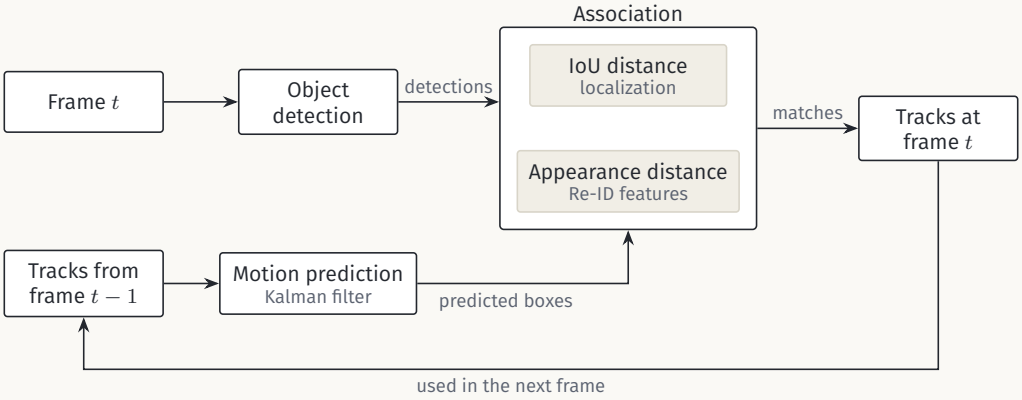


Tracker output on broadcast race footage (Eurosport). Each runner keeps the same ID from frame to frame.

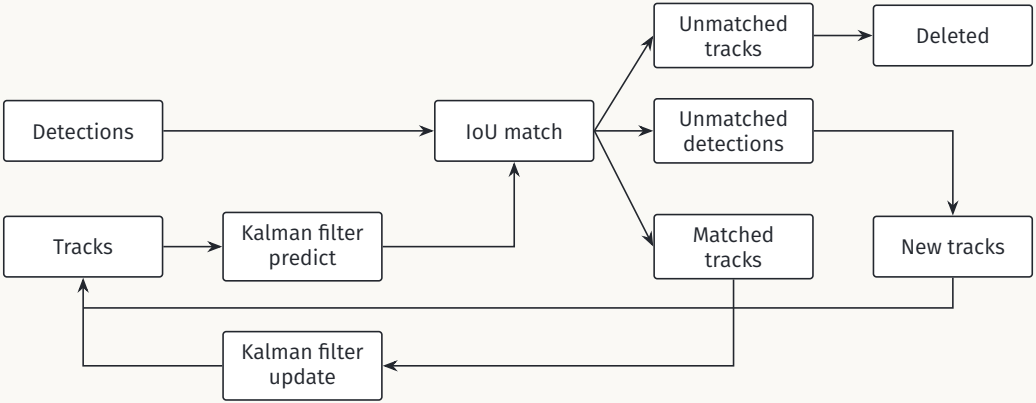
Tracking estimates trajectories over time

- **Multi-object tracking** (MOT) detects multiple objects in a video stream and estimates their spatial and temporal trajectories.
- **Tracking by detection** is currently the most effective paradigm for MOT: an object detection step is followed by a tracking step.
- The tracking step usually has two parts:
  1. A **motion model and state estimation** that predict the bounding boxes of existing tracklets in the following frames. A Kalman filter (KF) is the popular choice.
  2. **Association** of the new frame's detections with the current set of tracks, using either
     - the object's *localization*, mainly the intersection over union (IoU) between the predicted tracklet box and the detection box, or
     - an *appearance model* of the object, which turns association into a re-identification (Re-ID) task.
- Both cues are expressed as distances between tracks and detections.

The tracking-by-detection pipeline



Every track is matched, left unmatched, or created



Redrawn after https://www.mdpi.com/2076-3417/11/12/5619.

Tracking by detection, from video to trajectories

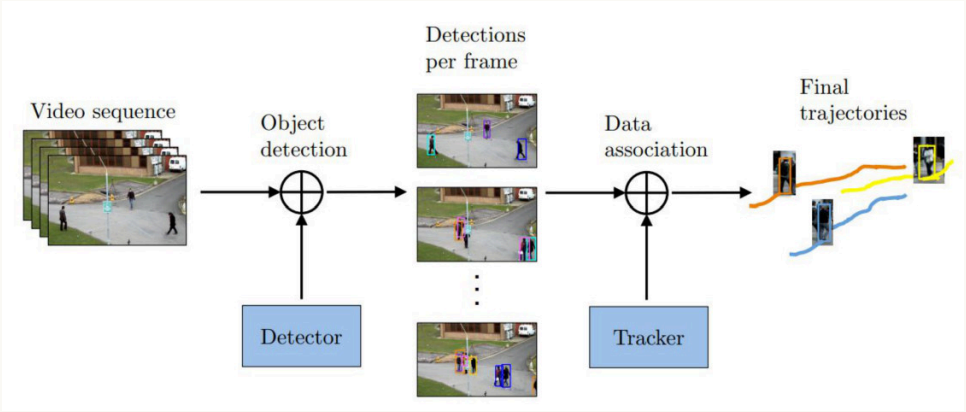


Figure from https://arxiv.org/abs/1411.7935.

PART 2

The Kalman filter

A bounding box described by seven quantities

- We describe a tracked bounding box by the state

$$\mathbf{x} = [\, x,\ y,\ s,\ r,\ \dot{x},\ \dot{y},\ \dot{s}\,]^{\mathsf{T}},$$

  where (x, y) is the box center, s its area, r its aspect ratio, and $\dot{x}, \dot{y}, \dot{s}$ the rates of change of x, y, and s.
- We assume the aspect ratio of an object stays constant while it is observed in a video, while the other quantities may change.
- The Kalman filter infers the first-order values $\dot{x}, \dot{y}, \dot{s}$ under two assumptions:
  - our sensors are noisy, and their output and noise can be modeled by Gaussians;
  - our initial prior on the state is also Gaussian.
- After an update, the next step can *predict* $x \leftarrow x + \dot{x}$, and likewise for y and s (with $\Delta t = 1$).
- New measurements then require updating our estimates of $\dot{x}$, $\dot{y}$, and $\dot{s}$.

The model behind the filter

The true state at time k evolves from the state at time $k-1$:

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$

- $\mathbf{F}_k$: the state-transition model, applied to the previous state $\mathbf{x}_{k-1}$.
- $\mathbf{B}_k$: the control-input model, applied to the control vector $\mathbf{u}_k$.
- $\mathbf{w}_k$: the process noise, drawn from a zero-mean multivariate normal distribution with covariance $\mathbf{Q}_k$: $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_k)$.

At time k we make an observation (measurement) $\mathbf{z}_k$ of the true state:

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

- $\mathbf{H}_k$: the observation model, which maps the true state space into the observed space.
- $\mathbf{v}_k$: the observation noise, zero-mean Gaussian white noise with covariance $\mathbf{R}_k$: $\mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R}_k)$.

After https://en.wikipedia.org/wiki/Kalman_filter.

What the filter keeps track of

The filter's state

- $\mathbf{x}_{k|k}$: the *a posteriori* state estimate at time k, given observations up to and including time k.
- $\mathbf{P}_{k|k}$: the *a posteriori* estimate covariance matrix, a measure of the estimated accuracy of the state estimate.

After https://en.wikipedia.org/wiki/Kalman_filter.

To estimate the internal state of a process from a sequence of noisy observations, we specify, for each time step k:

- $\mathbf{F}_k$, the state-transition model;
- $\mathbf{H}_k$, the observation model;
- $\mathbf{Q}_k$, the covariance of the process noise;
- $\mathbf{R}_k$, the covariance of the observation noise;
- sometimes $\mathbf{B}_k$, the control-input model, together with $\mathbf{u}_k$, the control vector that is its input.

Predict, then update

Predict

Predicted (*a priori*) state estimate

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \mathbf{x}_{k-1|k-1} + \mathbf{B}_k \mathbf{u}_k$$

Predicted (*a priori*) estimate covariance

$$\hat{\mathbf{P}}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^\mathsf{T} + \mathbf{Q}_k$$

Update

Innovation (pre-fit residual)	$\tilde{\mathbf{y}}_k = \mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k\|k-1}$
Innovation covariance	$\mathbf{S}_k = \mathbf{H}_k \hat{\mathbf{P}}_{k\|k-1} \mathbf{H}_k^\mathsf{T} + \mathbf{R}_k$
Optimal Kalman gain	$\mathbf{K}_k = \hat{\mathbf{P}}_{k\|k-1} \mathbf{H}_k^\mathsf{T} \mathbf{S}_k^{-1}$
Updated state estimate	$\mathbf{x}_{k\|k} = \hat{\mathbf{x}}_{k\|k-1} + \mathbf{K}_k \tilde{\mathbf{y}}_k$
Updated estimate covariance	$\mathbf{P}_{k\|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \hat{\mathbf{P}}_{k\|k-1}$
Post-fit residual	$\tilde{\mathbf{y}}_{k\|k} = \mathbf{z}_k - \mathbf{H}_k \mathbf{x}_{k\|k}$

After https://en.wikipedia.org/wiki/Kalman_filter.

PART 3

SORT

SORT in three stages

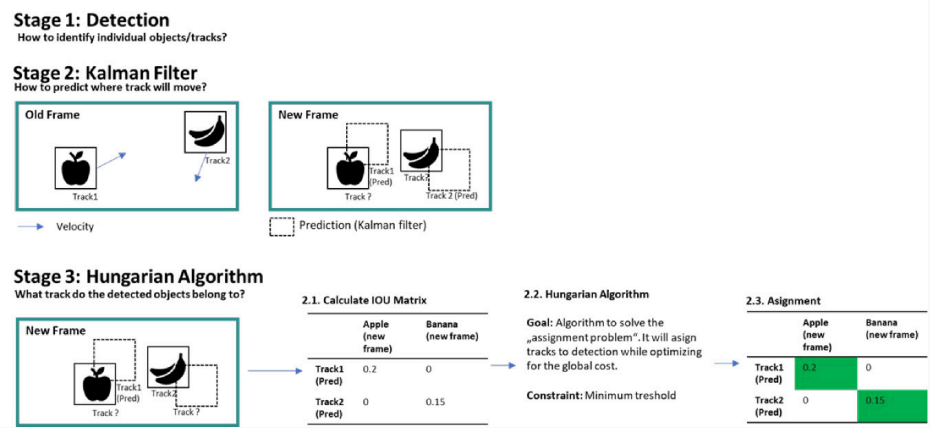


Figure by Daniel Pleus, https://www.linkedin.com/pulse/object-tracking-sort-deepsort-daniel-pleus/.

SORT is simple to implement

1. For each detected object, create a Kalman filter tracker and use it to predict where the object will be in the next frame.
2. Run the object detector on the new frame.
3. Match detected objects to trackers with the Hungarian algorithm, and update the trackers with that information.
4. Create new trackers for new objects.
5. Destroy trackers whose objects have not been seen recently (*stale* trackers).

In the code that follows

`linear_assignment` matches tracks to detections, and `KalmanBoxTracker` holds one object's state, with `predict` and `update` methods.

Hungarian algorithm for assignment

```python
def linear_assignment(cost_matrix):
  try:
    import lap
    _, x, y = lap.lapjv(cost_matrix, extend_cost=True)
    return np.array([[y[i],i] for i in x if i >= 0]) #
  except ImportError:
    from scipy.optimize import linear_sum_assignment
    x, y = linear_sum_assignment(cost_matrix)
    return np.array(list(zip(x, y)))
```

- The cost matrix has one entry per (track, detection) pair; SORT passes the negative IoU, so minimizing cost maximizes overlap.
- The result is the list of matched index pairs with the lowest total cost.
- The fast `lap` solver is used when installed; otherwise SciPy's solver.

State representation: from box corners to $\mathbf{z}$

```
def convert_bbox_to_z(bbox):
  """
  Takes a bounding box in the form [x1,y1,x2,y2] and returns z in the form
    [x,y,s,r] where x,y is the centre of the box and s is the scale/area and r is
    the aspect ratio
  """
  w = bbox[2] - bbox[0]
  h = bbox[3] - bbox[1]
  x = bbox[0] + w/2.
  y = bbox[1] + h/2.
  s = w * h    #scale is just area
  r = w / float(h)
  return np.array([x, y, s, r]).reshape((4, 1))
```

In math. From the corners (x_1, y_1, x_2, y_2), the measurement is $\mathbf{z} = [\, x,\ y,\ s,\ r \,]^\mathsf{T}$ with $w = x_2 - x_1$, $h = y_2 - y_1$, $x = x_1 + \frac{w}{2}$, $y = y_1 + \frac{h}{2}$, $s = wh$, and $r = \frac{w}{h}$.

Setting up the Kalman filter

```
class KalmanBoxTracker(object):
  """
  This class represents the internal state of individual tracked objects observed as bbox.
  """
  count = 0
  def __init__(self,bbox):
    """
    Initialises a tracker using initial bounding box.
    """
    #define constant velocity model
    self.kf = KalmanFilter(dim_x=7, dim_z=4)
    self.kf.F = np.array([[1,0,0,0,1,0,0],[0,1,0,0,0,1,0],[0,0,1,0,0,0,1],[0,0,0,1,0,0,0],
                          [0,0,0,0,1,0,0],[0,0,0,0,0,1,0],[0,0,0,0,0,0,1]])
    self.kf.H = np.array([[1,0,0,0,0,0,0],[0,1,0,0,0,0,0],[0,0,1,0,0,0,0],[0,0,0,1,0,0,0]])

    self.kf.R[2:,2:] *= 10.
    self.kf.P[4:,4:] *= 1000. #give high uncertainty to the unobservable initial velocities
    self.kf.P *= 10.
    self.kf.Q[-1,-1] *= 0.01
    self.kf.Q[4:,4:] *= 0.01

    self.kf.x[:4] = convert_bbox_to_z(bbox)
```

The model matrices in SORT

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

- $\mathbf{F}$, the state-transition model, is a constant-velocity model with $\Delta t = 1$: $x \leftarrow x + \dot{x}$, $y \leftarrow y + \dot{y}$, $s \leftarrow s + \dot{s}$, and r is unchanged.
- $\mathbf{H}$, the observation model, reads off $[x, y, s, r]$ from the seven-dimensional state.
- $\mathbf{R}$, the observation-noise covariance: `R[2:,2:] *= 10.` makes the measured area s and aspect ratio r noisier than the center.
- $\mathbf{P}$, the estimate covariance: `P[4:,4:] *= 1000.` gives high uncertainty to the unobservable initial velocities; `P *= 10.` scales the whole prior.
- $\mathbf{Q}$, the process-noise covariance: `Q[4:,4:] *= 0.01` and `Q[-1,-1] *= 0.01` keep the velocities changing slowly.
- The state $\mathbf{x}$ starts with its first four entries set to the measured box $\mathbf{z}$.

Updating a tracker with a matched detection

```
def update(self,bbox):
  """
  Updates the state vector with observed bbox.
  """
  self.time_since_update = 0
  self.history = []
  self.hits += 1
  self.hit_streak += 1
  self.kf.update(convert_bbox_to_z(bbox))
```

- Called when the tracker is matched to a detection.
- Resets the counter of frames without a match and increases the hit counters.
- `kf.update` runs the Kalman update step with the measurement $\mathbf{z}$ from the new box.

Predicting a tracker's next box

```
def predict(self):
  """
  Advances the state vector and returns the predicted bounding box estimate.
  """
  if((self.kf.x[6]+self.kf.x[2])<=0):
    self.kf.x[6] *= 0.0
  self.kf.predict()
  self.age += 1
  if(self.time_since_update>0):
    self.hit_streak = 0
  self.time_since_update += 1
  self.history.append(convert_x_to_bbox(self.kf.x))
  return self.history[-1]
```

- If the predicted area $s + \dot{s}$ would not be positive, the area velocity $\dot{s}$ is set to zero.
- `kf.predict` runs the Kalman predict step.
- A tracker that missed its last update loses its hit streak.
- The predicted state is converted back to a box and returned.

PART 4

Evaluation

Multiple object tracking accuracy (MOTA)

$$\text{MOTA} = 1 - \frac{\sum_t (m_t + fp_t + mme_t)}{\sum_t g_t}$$

- m_t: the number of misses (unmatched objects) in frame t.
- fp_t: the number of false positives (unmatched detections).
- mme_t: the number of mismatches, matches that contradict the match in the previous frame.
- g_t: the number of ground-truth objects present.

Multiple object tracking precision (MOTP)

$$\text{MOTP} = \frac{\sum_{i,t} d_{i,t}}{\sum_t c_t}$$

- $d_{i,t}$: the position error for matched object-hypothesis pair i in frame t, usually measured with IoU.
- c_t: the number of matches made in frame t.
- MOTP measures how well matched boxes are localized; MOTA measures how many errors the tracker makes.

IDF1 matches whole trajectories

- **IDF1** computes a **bijective** (one-to-one) mapping between the sets of ground-truth trajectories (gtTrajs) and predicted trajectories (prTrajs).
- **IDTPs** (identity true positives) are matches on the overlapping part, with valid detection thresholds, of trajectories that are matched together.
- **IDFNs** (identity false negatives) and **IDFPs** (identity false positives) are the remaining ground-truth detections (gtDets) and predicted detections (prDets), respectively: those in the non-overlapping sections of matched trajectories and those in trajectories that are not matched.
- **The matching** is chosen to optimize IDF1. For each possible match we count the IDFPs and IDFNs it would produce (from the non-overlapping sections), and for unmatched trajectories we count their prDets and gtDets. The Hungarian algorithm then selects the trajectory matches that minimize the sum of IDFPs and IDFNs.

IDF1: identification precision, recall, and F1

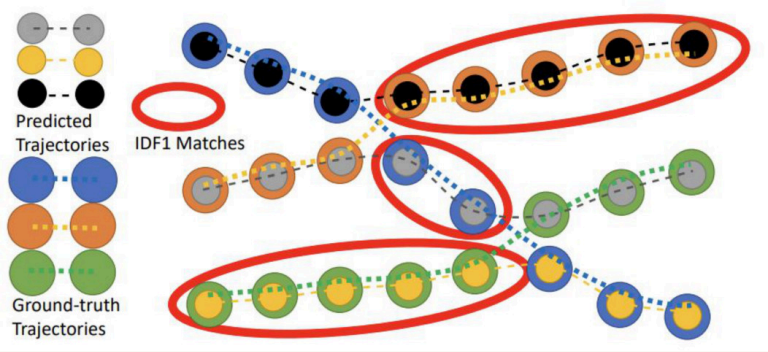
$$\text{ID-Recall} = \frac{|\text{IDTP}|}{|\text{IDTP}| + |\text{IDFN}|}$$

$$\text{ID-Precision} = \frac{|\text{IDTP}|}{|\text{IDTP}| + |\text{IDFP}|}$$

$$\text{IDF1} = \frac{|\text{IDTP}|}{|\text{IDTP}| + 0.5\ |\text{IDFN}| + 0.5\ |\text{IDFP}|}$$

IDF1 is the harmonic mean of ID-Recall and ID-Precision.

IDF1 can produce unintuitive matches

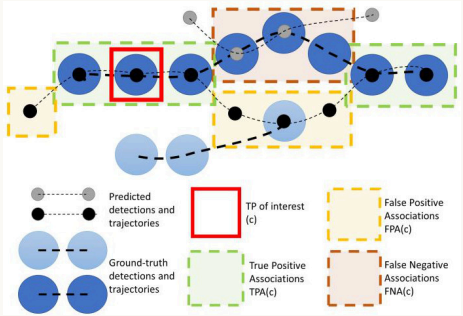


A tracking example showing how the single best trajectory matching, as performed by IDF1, can result in unintuitive matches between trajectories.

Luiten et al., *HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking*, IJCV 2021, Fig. 10.

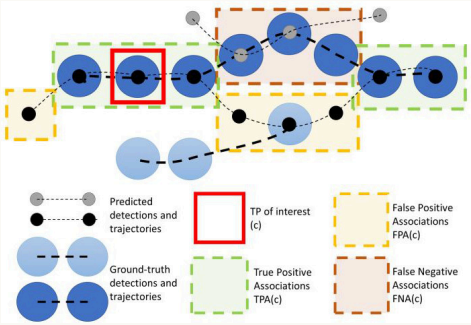
Higher order tracking accuracy (HOTA)

HOTA computes a **bijective** (one-to-one) mapping between the sets of gtDets and prDets in each frame. Given the matching, for each matched pair that is a true positive (TP) at a given IoU threshold, we compute the following quantities.



Luiten et al., *HOTA*, https://arxiv.org/abs/2009.07736.

Three kinds of association for one true positive



For the TP of interest c (red):

- **TPAs** (green) are matches with the same prID and the same gtID as c.
- **FPAs** (yellow) have the same prID but a different gtID, or none.
- **FNAs** (orange) have the same gtID but a different prID, or none.
- Here c has five TPAs, four FPAs, and three FNAs.

The question they answer

For the matched TP c, how accurate is the alignment between its gtTraj (large dark blue circles) and its prTraj (small black circles)?

Luiten et al., *HOTA*, Fig. 2. Implementation: https://github.com/JonathonLuiten/TrackEval/blob/master/trackeval/metrics/hota.py.

Defining the associations

For a true positive c, with $\mathrm{prID}(\cdot)$ and $\mathrm{gtID}(\cdot)$ the predicted and ground-truth identities:

$$\mathrm{TPA}(c) = \{k\}, \qquad k \in \{\mathrm{TP} \mid \mathrm{prID}(k) = \mathrm{prID}(c) \wedge \mathrm{gtID}(k) = \mathrm{gtID}(c)\}$$

$$\mathrm{FNA}(c) = \{k\}, \qquad k \in \{\mathrm{TP} \mid \mathrm{prID}(k) \neq \mathrm{prID}(c) \wedge \mathrm{gtID}(k) = \mathrm{gtID}(c)\} \cup \{\mathrm{FN} \mid \mathrm{gtID}(k) = \mathrm{gtID}(c)\}$$

$$\mathrm{FPA}(c) = \{k\}, \qquad k \in \{\mathrm{TP} \mid \mathrm{prID}(k) = \mathrm{prID}(c) \wedge \mathrm{gtID}(k) \neq \mathrm{gtID}(c)\} \cup \{\mathrm{FP} \mid \mathrm{prID}(k) = \mathrm{prID}(c)\}$$

The HOTA score

The association accuracy of a true positive c:

$$\mathcal{A}(c) = \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}$$

HOTA at a localization threshold α:

$$\mathrm{HOTA}_\alpha = \sqrt{\frac{\sum_{c \in \{\mathrm{TP}\}} \mathcal{A}(c)}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}}$$

HOTA integrates over all thresholds:

$$\mathrm{HOTA} = \int_0^1 \mathrm{HOTA}_\alpha \, d\alpha \approx \frac{1}{19} \sum_{\alpha \in \{0.05,\, 0.1,\, \ldots,\, 0.9,\, 0.95\}} \mathrm{HOTA}_\alpha$$

Summary

- **Tracking by detection**: detect objects in each frame, predict where existing tracks will be, and associate detections with tracks by IoU or appearance.
- **Kalman filter**: a Gaussian state estimator that alternates *predict* and *update*; in SORT the state is $[x, y, s, r, \dot{x}, \dot{y}, \dot{s}]$.
- **SORT**: one Kalman tracker per object, Hungarian matching, new trackers for new objects, and deletion of stale ones.
- **Metrics**: MOTA counts errors, MOTP measures localization, IDF1 scores identity over whole trajectories, and HOTA combines detection and association accuracy in one score.